Embedded System Programming: Coding with C/C++

Learning Outcome 1 (continued):

Develop programs in Assembly and C to perform specific tasks.

1

Possible Software Tools (Assembler/Compiler) for ARM Processors

- Keil Compiler: free and paid version available
- mbed (online compiler): requires internet connection

Open

2

Page 3

INTRODUCTION TO EMBEDDED PROGRAMMING IN C/C++

Open

3

Page 4

Reasons to Use C

- Legacy Code
  - Linux, most open source applications are in C
- Smaller, simpler to write compiler
  - Programming embedded systems, often only have a C compiler
- Performance
  - Typically 50x faster than interpreted Java

Open

4

Page 5

Hello World in C

```
#include <stdio.h>

void main()
{
  printf("Hello, world!\n");
}
```

Program mostly a collection of functions

"main" function special: the entry point

"void" qualifier indicates function does not return anything

I/O performed by a library function: not included in the language

Open

5

Page 6

C Syntax and Hello World

#include inserts another file. ".h" files are called "header" files. They contain stuff needed to interface to libraries and code in other ".c" files.

This is a comment. The compiler ignores this.

```
#include <stdio.h>
/* The simplest C Program */
int main(int argc, char **argv)
{
  printf("Hello World\n");
  return 0;
}
```

The main() function is always where your program starts running.

Blocks of code ("lexical scopes") are marked by { … }

Return '0' from this function

Print out a message. '\n' means "new line".

Open Open

6

Page 7

Basic C program structure

```
#include "STM32L1xx.h"      /* I/O port/register names/addresses for the STM32L1xx microcontrollers */

/* Global variables – accessible by all functions */
int count, bob;             //global (static) variables – placed in RAM

/* Function definitions*/
int function1(char x) {     //parameter x passed to the function, function returns an integer value
  int i,j;                  //local (automatic) variables – allocated to stack or registers
  -- instructions to implement the function
}

/* Main program */
void main(void) {
   unsigned char sw1;       //local (automatic) variable (stack or registers)
   int k;                   //local (automatic) variable (stack or registers)
  /* Initialization section */
   -- instructions to initialize  variables, I/O ports, devices, function registers
  /* Endless loop */
   while (1) {              //Can also use:  for(;;) {
    -- instructions to be repeated
   } /* repeat forever */
  }
```

Declare local variables

Initialize variables/devices

Body of the program

Open

7

Page 8

C statement types

- Simple variable assignments
  - Includes input/output data transfers
- Arithmetic operations
- Logical/shift operations
- Control structures
  - IF, WHEN, FOR, SELECT
- Function calls
  - User-defined and/or library functions

Open

8

C Data Type Examples

```
int i;                  Integer
int *j, k;              j: pointer to integer, int k
unsigned char *ch;      ch: pointer to unsigned char
float f[10];            Array of 10 floats
char nextChar(int, char*);      2-arg function
int a[3][5][10];        Array of three arrays of five …
int *func1(float);      function returning int *
int (*func2)(void);     pointer to function returning int
```


Program variables

- A *variable* is an addressable storage location to information to be used by the program
  - Each variable must be *declared* to indicate size and type of information to be stored, plus name to be used to reference the information

    int x,y,z; //declares 3 variables of type "int"

    char a,b; //declares 2 variables of type "char"
  - Space for variables may be allocated in registers, RAM, or ROM/Flash (for constants)
  - Variables can be *automatic* or *static*

What is "Memory"?

Memory is like a big table of numbered slots where bytes can be stored.

The number of a slot is its Address.
One byte Value can be stored in each slot.

Some "logical" data values span more than one slot, like the character string "Hello\n"

A Type names a logical meaning to a span of memory. Some simple types are:

`char`	a single character (1 slot)
`char [10]`	an array of 10 characters
`int`	signed 4 byte integer
`float`	4 byte floating point
`int64_t`	signed 8 byte integer

Addr	Value
0	
1	
2	
3	
4	'H' (72)
5	'e' (101)
6	'l' (108)
7	'l' (108)
8	'o' (111)
9	'\n' (10)
10	'\0' (0)
11	
12	

Open

11

What is a Variable?

A Variable names a place in memory where you store a Value of a certain Type.

You first Define a variable by giving it a name and specifying the type, and optionally an initial value

```
char x;
char y='e';
```

Initial value of x is undefined

Initial value

Name

Type is single character (char)

The compiler puts them somewhere in memory.

Symbol	Addr	Value
	0	
	1	
	2	
	3	
x	4	?
y	5	'e' (101)
	6	
	7	
	8	
	9	
	10	
	11	
	12	

Open Open

12

Multi-byte Variables

Different types consume different amounts of memory. Most architectures store data on "word boundaries", or even multiples of the size of a primitive data type (int, char)

```
char x;
char y='e';
int z = 0x01020304;
```

0x means the constant is written in hex

padding

An int consumes 4 bytes

Symbol	Addr	Value
	0	
	1	
	2	
	3	
x	4	?
y	5	'e' (101)
	6	
	7	
z	8	4
	9	3
	10	2
	11	1
	12	

Open

13

Automatic variables

- Declare within a function/procedure
- Variable is visible (has *scope*) only within that function
  - Space for the variable is allocated on the system *stack* when the procedure is entered
    - Deallocated, to be re-used, when the procedure is exited
  - If only 1 or 2 variables, the compiler may allocate them to registers within that procedure, instead of allocating memory.
  - Values are not retained between procedure calls

Open

14

Automatic variable example

```
void delay () {
 int i,j;    //automatic variables – visible only within delay()
 for (i=0; i<100; i++) {          //outer loop
    for (j=0; j<20000; j++) {     //inner loop
    }                             //do nothing
 }
}
```

Variables must be initialized each time the procedure is entered since values are not retained when the procedure is exited.

Static variables

- Retained for use throughout the program in RAM locations that are *not reallocated* during program execution.
- Declare either within or outside of a function
  - If declared outside a function, the variable is *global* in scope, i.e. known to all functions of the program
    - Use "normal" declarations. Example: *int count;*
  - If declared within a function, insert key word *static* before the variable definition. The variable is *local* in scope, i.e. known only within this function.

```
static unsigned char bob;
static int pressure[10];
```

Static variable example

```
unsigned char count;  //global variable is static – allocated a fixed RAM location
                      //count can be referenced by any function
void math_op () {
 int i;               //automatic variable – allocated space on stack when function entered
 static int j;        //static variable – allocated a fixed RAM location to maintain the value
 if (count == 0)      //test value of global variable count
     j = 0;           //initialize static variable j first time math_op() entered
 i = count;           //initialize automatic variable i each time math_op() entered
 j = j + i;           //change static variable j – value kept for next function call
}                     //return & deallocate space used by automatic variable i

void main(void) {
 count = 0;           //initialize global variable count
 while (1) {
  math_op();
  count++;            //increment global variable count
 }
```

Open

17

Comparison and Mathematical Operators

```
+  plus
-  minus
*  mult
/  divide
%  modulo
```

```
== equal to
<  less than
<= less than or equal
>  greater than
>= greater than or equal
!= not equal
&& logical and
|| logical or
!  logical not
```

```
&  bitwise and
|  bitwise or
^  bitwise xor
~  bitwise not
<< shift left
>> shift right
```

The rules of precedence are clearly defined but often difficult to remember or non-intuitive. When in doubt, add parentheses to make it explicit. For oft-confused cases, the compiler will give you a warning

Beware division:
- If second argument is integer, the result will be integer (rounded):
  5 / 10 → 0 *whereas* 5 / 10.0 → 0.5
- Division by 0 will cause a FPE

Don't confuse & and &&..
1 & 2 → 0 *whereas* 1 && 2 → <true>

Open

18

Arithmetic operations

- C examples – with standard arithmetic operators

```
int i, j, k;      // 32-bit signed integers
uint8_t m,n,p;    // 8-bit unsigned numbers
i = j + k;        // add 32-bit integers
m = n - 5;        // subtract 8-bit numbers
j = i * k;        // multiply 32-bit integers
m = n / p;        // quotient of 8-bit divide
m = n % p;        // remainder of 8-bit divide
i = (j + k) * (i - 2);  //arithmetic expression
```

*, /, % are higher in precedence than +, - (higher precedence applied 1st)
Example: j * k + m / n = (j * k) + (m / n)

Floating-point formats are not directly supported by Cortex-M3 CPUs.

Relational Operators

- Test relationship between two variables/expressions

Test	TRUE condition	Notes
(m == b)	m equal to b	Double =
(m != b)	m not equal to b	
(m < b)	m less than b	1
(m <= b)	m less than or equal to b	1
(m > b)	m greater than b	1
(m >= b)	m greater than or equal to b	1
(m)	m non-zero	
(1)	always TRUE	
(0)	always FALSE	

1. Compiler uses signed or unsigned comparison, in accordance with data types

Example:

unsigned char a,b;
int j,k;
if (a < b) – unsigned
if (j > k) - signed

Common error

- Note that == is a relational operator, whereas = is an assignment operator.

```
if ( m == n)  //tests equality of values of variables m and n
if (m = n)    //assigns value of n to variable m, and then
              //tests whether that value is TRUE (non-zero)
```

The second form is a common error (omitting the second equal sign), and usually produces unexpected results, namely a TRUE condition if n is 0 and FALSE if n is non-zero.

Boolean operators

- Boolean operators && (AND) and || (OR) produce TRUE/FALSE results when testing multiple TRUE/FALSE conditions

```
if ((n > 1) && (n < 5))   //test for n between 1 and 5
if ((c = 'q') || (c = 'Q')) //test c = lower or upper case Q
```

- Note the difference between Boolean operators &&, || and bitwise logical operators &, |

```
if ( k && m)   //test if k and m both TRUE (non-zero values)
if ( k & m)    //compute bitwise AND between m and n,
               //then test whether the result is non-zero (TRUE)
```

Bit-parallel logical operators

Bit-parallel (bitwise) logical operators produce n-bit results of the corresponding logical operation:

& (AND) | (OR) ^ (XOR) ~ (Complement)

```
C = A & B;        A   0 1 1 0 0 1 1 0
(AND)             B   1 0 1 1 0 0 1 1
                  C   0 0 1 0 0 0 1 0

C = A | B;        A   0 1 1 0 0 1 0 0
(OR)              B   0 0 0 1 0 0 0 0
                  C   0 1 1 1 0 1 0 0

C = A ^ B;        A   0 1 1 0 0 1 0 0
(XOR)             B   1 0 1 1 0 0 1 1
                  C   1 1 0 1 0 1 1 1

B = ~A;           A   0 1 1 0 0 1 0 0
(COMPLEMENT)      B   1 0 0 1 1 0 1 1
```

Open

23

Bit set/reset/complement/test

- Use a "mask" to select bit(s) to be altered

```
C = A & 0xFE;     A   a b c d e f g h
                0xFE  1 1 1 1 1 1 1 0   Clear selected bit of A
                  C   a b c d e f g 0

C = A & 0x01;     A   a b c d e f g h
                0xFE  0 0 0 0 0 0 0 1   Clear all but the selected bit of A
                  C   0 0 0 0 0 0 0 h

C = A | 0x01;     A   a b c d e f g h
                0x01  0 0 0 0 0 0 0 1   Set selected bit of A
                  C   a b c d e f g 1

C = A ^ 0x01;     A   a b c d e f g h
                0x01  0 0 0 0 0 0 0 1   Complement selected bit of A
                  C   a b c d e f g h'
```

Open

24

Bit examples for input/output

- Create a "pulse" on bit 0 of PORTA (assume bit is initially 0)

```
PORTA = PORTA | 0x01;  //Force bit 0 to 1
PORTA = PORTA & 0xFE;  //Force bit 0 to 0
```

- Examples:

```
if ( (PORTA & 0x80) != 0 )  //Or: ((PORTA & 0x80) == 0x80)
    bob();                  // call bob() if bit 7 of PORTA is 1
c = PORTB & 0x04;           // mask all but bit 2 of PORTB value
if ((PORTA & 0x01) == 0)    // test bit 0 of PORTA
    PORTA = c | 0x01;       // write c to PORTA with bit 0 set to 1
```


Shift operators

Shift operators:
x >> y (right shift operand x by y bit positions)
x << y (left shift operand x by y bit positions)
Vacated bits are filled with 0's.
Shift right/left fast way to multiply/divide by power of 2

```
B = A << 3;              A   1 0 1 0 1 1 0 1
(Left shift 3 bits)      B   0 1 1 0 1 0 0 0

B = A >> 2;              A   1 0 1 1 0 1 0 1
(Right shift 2 bits)     B   0 0 1 0 1 1 0 1

B = '1';                 B = 0 0 1 1 0 0 0 1 (ASCII 0x31)
C = '5';                 C = 0 0 1 1 0 1 0 1 (ASCII 0x35)
D = (B << 4) | (C & 0x0F);
       (B << 4)      = 0 0 0 1 0 0 0 0
     (C & 0x0F)      = 0 0 0 0 0 1 0 1
             D       = 0 0 0 1 0 1 0 1 (Packed BCD 0x15)
```

Expressions and Evaluation

Expressions combine Values using Operators, according to precedence.

```
1 + 2 * 2      → 1 + 4      → 5
(1 + 2) * 2    → 3 * 2      → 6
```

Symbols are evaluated to their Values before being combined.

```
int x=1;
int y=2;
x + y * y      → x + 2 * 2      → x + 4      → 1 + 4      → 5
```

Comparison operators are used to compare values.
In C, 0 means “false”, and *any other value* means “true”.

```
int x=4;
(x < 5)                 → (4 < 5)              → <true>
(x < 4)                 → (4 < 4)              → 0
((x < 5) || (x < 4))    → (<true> || (x < 4))  → <true>
```

Not evaluated because first clause was true

Assignment Operators

```
x = y    assign y to x
x++      post-increment x
++x      pre-increment x
x--      post-decrement x
--x      pre-decrement x
```

```
x += y   assign (x+y) to x
x -= y   assign (x-y) to x
x *= y   assign (x*y) to x
x /= y   assign (x/y) to x
x %= y   assign (x%y) to x
```

Note the difference between ++x and x++:

```
int x=5;
int y;
y = ++x;
/* x == 6, y == 6 */
```

```
int x=5;
int y;
y = x++;
/* x == 6, y == 5 */
```

Don’t confuse = and ==!

```
int x=5;
if (x==6)   /* false */
{
  /* ... */
}
/* x is still 5 */
```

```
int x=5;
if (x=6)   /* always true */
{
  /* x is now 6 */
}
/* ... */
```

"Pointers"

This is exactly how "pointers" work.

"address of" or reference operator: &
"memory_at" or dereference operator: *

A "pointer type": pointer to char

```
void f(address_of_char p)
{
  memory_at[p] = memory_at[p] - 32;
}

char y = 101;       /* y is 101 */
f(address_of(y));  /* i.e. f(5) */
/* y is now 101-32 = 69 */
```

```
void f(char * p)
{
  *p = *p - 32;
}

char y = 101;       /* y is 101 */
f(&y);              /* i.e. f(5) */
/* y is now 101-32 = 69 */
```

Pointers are used in C for many other purposes:
- Passing large objects without copying them
- Accessing dynamically allocated memory
- Referring to functions

Open

29

Arrays

Arrays in C are composed of a particular type, laid out in memory in a repeating pattern. Array elements are accessed by stepping forward in memory from the base of the array by a multiple of the element size.

```
/* define an array of 10 chars */
char x[5] = {'t','e','s','t','\0'};

/* accessing element 0 */
x[0] = 'T';

/* pointer arithmetic to get elt 3 */
char elt3 = *(x+3);  /* x[3] */

/* x[0] evaluates to the first element;
 * x evaluates to the address of the
 * first element, or &(x[0]) */

/* 0-indexed for loop idiom */
#define COUNT 10
char y[COUNT];
int i;
for (i=0; i<COUNT; i++) {
  /* process y[i] */
  printf("%c\n", y[i]);
}
```

Brackets specify the count of elements. Initial values optionally set in braces.

Arrays in C are 0-indexed (here, 0..9)

x[3] == *(x+3) == 't' (*NOT* 's'!)

What's the difference between char x[] and char *x?

For loop that iterates from 0 to COUNT-1. Memorize it!

Symbol	Addr	Value
char x [0]	100	't'
char x [1]	101	'e'
char x [2]	102	's'
char x [3]	103	't'
char x [4]	104	'\0'

Open

30

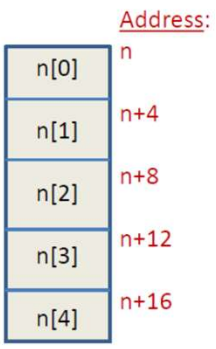
Variable arrays

- An *array* is a set of data, stored in consecutive memory locations, beginning at a named address
  - Declare array name and number of data elements, N
  - Elements are "indexed", with indices [0 .. N-1]

int n[5]; //declare array of 5 "int" values

n[3] = 5; //set value of 4th array element

Note: Index of first element is always 0.



Open

31

User-Defined Structure Types

- Use struct to group data
- Dot (.) operator to access fields of a struct
- Fields are accessible everywhere (no way to make them private)

```
typedef struct {
  char name[10];
  int count;
} Tally;
```

32

Open

32

C Structures

- A struct is an object with named fields:

```
struct {
  char *name;
  int x, y;
  int h, w;
} box;
```

- Accessed using "dot" notation:

```
box.x = 5;
box.y = 2;
```

Open

33

C control structures

- Control order in which instructions are executed (program flow)
- Conditional execution
  - Execute a set of statements if some condition is met
  - Select one set of statements to be executed from several options, depending on one or more conditions
- Iterative execution
  - Repeated execution of a set of statements
    - A specified number of times, or
    - Until some condition is met, or
    - While some condition is true

Open

34

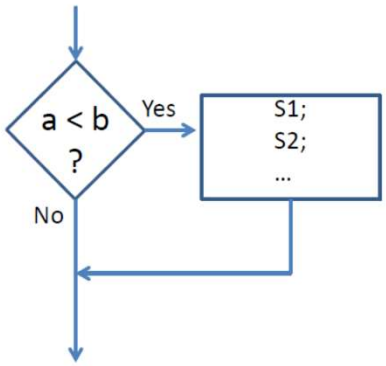
Page 35

IF-THEN structure

- Execute a set of statements if and only if some condition is met

TRUE/FALSE condition

```
if (a < b)
  {
   statement s1;
   statement s2;
   ....
  }
```



Open

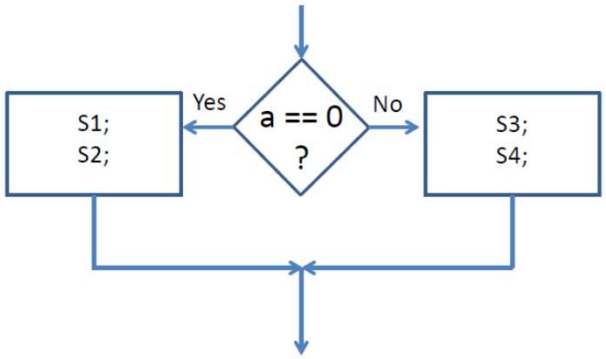
35

Page 36

IF-THEN-ELSE structure

- Execute one set of statements if a condition is met and an alternate set if the condition is not met.

```
if (a == 0)
  {
   statement s1;
   statement s2;
  }
else
 {
  statement s3;
  statement s4:
 }
```



Open

36

Page 37

Ambiguous ELSE association

```
if (n > 0)
    if (a > b)
        z = a;
else              //else goes with nearest previous "if" (a > b)
    z = b;

if (n > 0) {
    if (a > b)
        z = a;
} else  {         //else goes with first "if" (n > 0)
    z = b;
}
```

Braces force proper association

Open

37

Page 38

Multiple ELSE-IF structure

- Multi-way decision, with expressions evaluated in a specified order

```
if (n == 1)
   statement1;    //do if n == 1
else if (n == 2)
   statement2;   //do if n == 2
else if (n == 3)
   statement3;  //do if n == 3
else
   statement4;  //do if any other value of n (none of the above)
```

Any "statement" above can be replaced with a set of statements: {s1; s2; s3; ...}

Open

38

Page 39

SWITCH statement

- Compact alternative to ELSE-IF structure, for multi-way decision that tests one variable or expression for a number of constant values

```
/* example equivalent to that on preceding slide */
 switch ( n) {        //n is the variable to be tested
     case 0:  statement1;   //do if n == 0
     case 1:  statement2;  // do if n == 1
     case 2:  statement3;  // do if n == 2
     default: statement4;  //if for any other n value
}
```

Any "statement" above can be replaced with a set of statements: {s1; s2; s3; ...}

Open

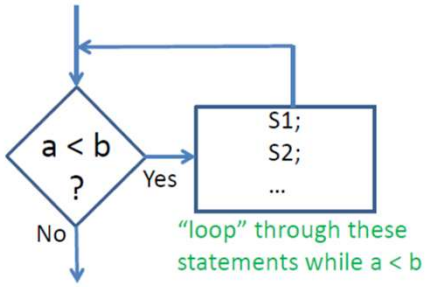
39

Page 40

WHILE loop structure

- Repeat a set of statements (a "loop") as long as some condition is met

```
while (a < b)
  {
    statement s1;
    statement s2;
    ....
  }
```



Something must eventually cause a >= b, to exit the loop

Open

40

Page 41

WHILE loop example

```
#define MAX_ALLOWED 128
#define LIGHT_ON 1
#define LIGHT_OFF 0


while (AD_PORT <= MAX_ALLOWED)
 {
   LIGHT_PORT = LIGHT_ON;
   delay();
   LIGHT_PORT = LIGHT_OFF;
   delay();
 }
```

Open

41

Page 42

The “while” loop

Situation requiring the “while” loop.

```
loop:
  if (condition) {
    statements;
    goto loop;
  }
```

⇨

```
while (condition) {
  statements;
}
```

```
float pow(float x, uint exp)
{
  int i=0;
  float result=1.0;
  while (i < exp) {
    result = result * x;
    i++;
  }
  return result;
}

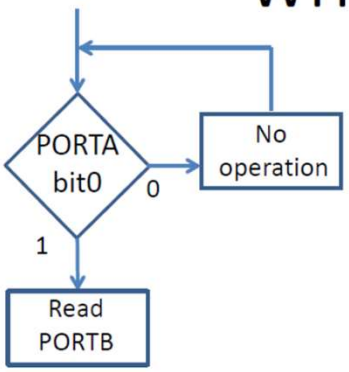
int main(int argc, char **argv)
{
  float p;
  p = pow(10.0, 5);
  printf("p = %f\n", p);
  return 0;
}
```

Open

42

Page 43

WHILE example



Wait for a 1 to be applied
to bit 0 of GPIOA
and then read GPIOB

```
while ( (GPIOA->IDR & 0x0001) == 0)  // test bit 0 of GPIOA
   {}                                // do nothing & repeat if bit is 0
c = GPIOB->IDR;                      // read GPIOB after above bit = 1
```

Open

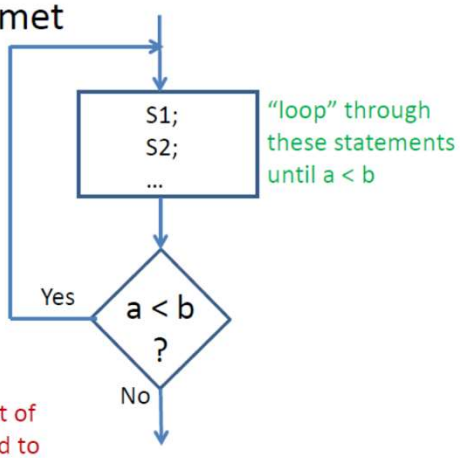
43

Page 44

DO-WHILE loop structure

- Repeat a set of statements (one "loop")
  until some condition is met

```
do
  {
   statement s1;
   statement s2;
   ....
  }
while (a < b);
```



The condition is tested after executing the set of statements, so the statements are guaranteed to execute at least once.

Open

44

Page 45

DO-WHILE example

```
#define MAX_ALLOWED 128
#define LIGHT_ON 1
#define LIGHT_OFF 0

do {
   LIGHT_PORT = LIGHT_ON;
   delay();
   LIGHT_PORT = LIGHT_OFF;
   delay();
} while (AD_PORT <= MAX_ALLOWED);
```

Open

45

Page 46

DO-WHILE example

```
/* Add two 200-element arrays. */
int M[200],N[200],P[200];
int k;

/* Method 1 – using DO-WHILE */
k = 0;                          //initialize counter/index
do {
   M[k] = N[k] + P[k];          //add k-th array elements
   k = k + 1;                   //increment counter/index
} while (k < 200);              //repeat if k less than 200

/* Method 2 – using WHILE loop
k = 0;                          //initialize counter/index
while (k < 200} {               //execute the loop if k less than 200
   M[k] = N[k] + P[k];          //add k-th array elements
   k = k + 1;                   //increment counter/index
}
```

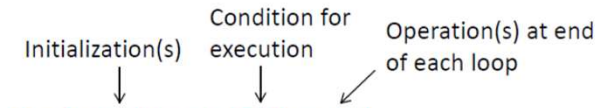
Open

46

Page 47

FOR loop structure

- Repeat a set of statements (one "loop") while some condition is met
  - often a given # of iterations




```
for (m = 0; m < 200; m++)
  {
    statement s1;
    statement s2;
  }
```

Open

47

Page 48

FOR loop structure

- FOR loop is a more compact form of the WHILE loop structure

```
/* execute loop 200 times */
for (m = 0; m < 200; m++)
  {
    statement s1;
    statement s2;
  }
```

```
/* equivalent WHILE loop */
m = 0;  //initial action(s)
while (m < 200)  //condition test
  {
    statement s1;
    statement s2;
    m = m + 1;  //end of loop action
  }
```

Open

48

Page 49

FOR structure example

```
/* Read 100 16-bit values from GPIOB into array C */
/* Bit 0 of GPIOA (PA0) is 1 if data is ready, and 0 otherwise */
uint16_t  c[100];
uint16_t  k;

for (k = 0; k < 200; k++) {
      while ((GPIOA->IDR & 0x01) == 0)   //repeat until PA0 = 1
         {}                              //do nothing if PA0 = 0
      c[k] = GPIOB->IDR;                 //read data from PB[15:0]
}
```

Open

49

Page 50

FOR structure example

```
/* Nested FOR loops to create a time delay */

for (i = 0; i < 100; i++) {          //do outer loop 100 times
   for (j = 0; j < 1000; j++) {      //do inner loop 1000 times
    }                                //do "nothing" in inner loop
}
```

Open

50

The "for" loop

The "for" loop is just shorthand for this "while" loop structure.

```
float pow(float x, uint exp)
{
  float result=1.0;
  int i;
  i=0;
  while (i < exp) {
    result = result * x;
    i++;
  }
  return result;
}

int main(int argc, char **argv)
{
  float p;
  p = pow(10.0, 5);
  printf("p = %f\n", p);
  return 0;
}
```



```
float pow(float x, uint exp)
{
  float result=1.0;
  int i;
  for (i=0; (i < exp); i++) {
    result = result * x;
  }
  return result;
}

int main(int argc, char **argv)
{
  float p;
  p = pow(10.0, 5);
  printf("p = %f\n", p);
  return 0;
}
```

Open

51

Summary of C Statements

- Expression
- Conditional
  - if (expr) { … } else {…}
  - switch (expr) { case c1: case c2: … }
- Iteration
  - while (expr) { … } zero or more iterations
  - do … while (expr) at least one iteration
  - for (init ; valid ; next) { … }
- Jump
  - goto label
  - **continue;** go to start of loop
  - **break;** exit loop or switch
  - **return expr;** return from function

Open

52

Page 53

C functions

- Functions partition large programs into a set of smaller tasks
  - Helps manage program complexity
  - Smaller tasks are easier to design and debug
  - Functions can often be reused instead of starting over
  - Can use of "libraries" of functions developed by 3rd parties, instead of designing your own

Open

53

Page 54

C functions

- A function is "called" by another program to perform a task
  - The *function* may return a result to the caller
  - One or more arguments may be passed to the function/procedure

Open

54

Page 55

Function definition

Type of value to be returned to the caller*

Parameters passed by the caller

```
int math_func (int k; int n)
{
 int j;              //local variable
 j = n + k - 5;      //function body
 return(j);          //return the result
}
```

* If no return value, specify "void"

Open

55

Page 56

Function arguments

- Calling program can pass information to a function in two ways
  - By value: pass a constant or a variable value
    - function can use, but not modify the value
  - By reference: pass the address of the variable
    - function can both read and update the variable
  - Values/addresses are typically passed to the function by pushing them onto the system stack
    - Function retrieves the information from the stack

Open

56

Page 57

Example – pass by value

```
/* Function to calculate x2 */
int square ( int x ) {  //passed value is type int, return an int value
   int y;               //local variable – scope limited to square
   y = x * x;           //use the passed value
   return(x);           //return the result
}

void main {
  int k,n;            //local variables – scope limited to main
  n = 5;
  k = square(n);     //pass value of n, assign n-squared to k
  n = square(5);     // pass value 5, assign 5-squared to n
}
```

Open

57

Page 58

Example – pass by reference

```
/* Function to calculate x2 */
void square ( int x, int *y ) {  //value of x, address of y
  *y = x * x;                    //write result to location whose address is y
}

void main {
  int k,n;              //local variables – scope limited to main
  n = 5;
  square(n, &k);   //calculate n-squared and put result in k
  square(5, &n);    // calculate 5-squared and put result in n
}
```

In the above, *main* tells *square* the location of its local variable, so that *square* can write the result to that variable.

Open

58

Example – receive serial data bytes

```
/* Put string of received SCI bytes into an array */
Int rcv_data[10];          //global variable array for received data
Int rcv_count;             //global variable for #received bytes

void SCI_receive ( ) {
  while ( (SCISR1 & 0x20) == 0) {}  //wait for new data (RDRF = 1)
  rcv_data[rcv_count] = SCIDRL;   //byte to array from SCI data reg.
  rcv_count++;                      //update index for next byte
}
```

Other functions can access the received data from the global variable array rcv_data[].

Open

59